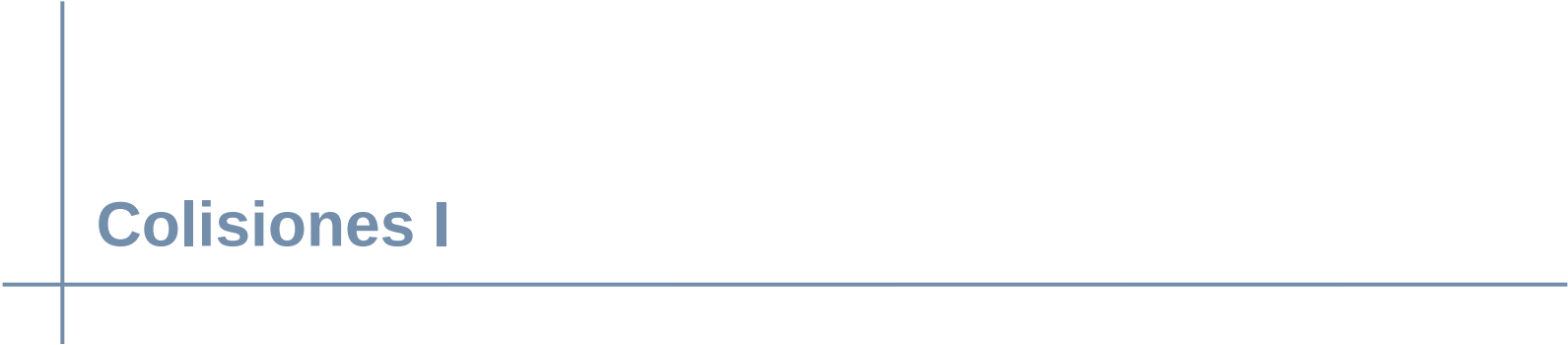




Colisiones I

R. Fernández
L. García

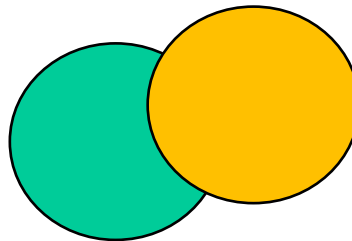


Colisiones





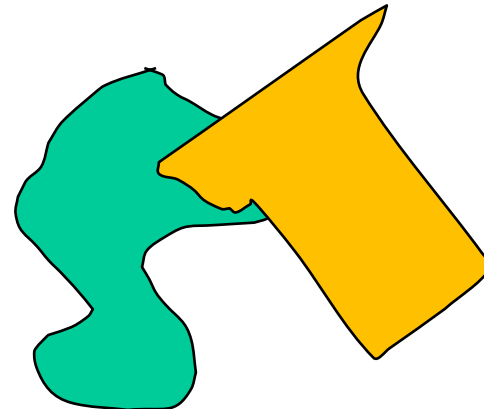
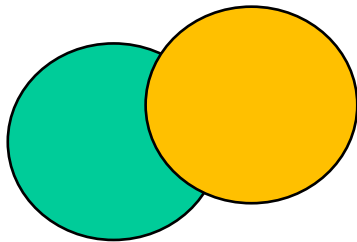
- ❑ Tenemos dos problemas a resolver:
 - ❑ Detección de colisiones
 - ❑ Gestión de colisiones
- ❑ La **detección de colisiones** consiste en detectar cuando se produce una intersección entre dos objetos. Es decir, cuando dos objetos ocupan el mismo espacio.



- ❑ La **gestión de colisiones** consiste en, una vez detectada la colisión, encargarnos de que se produce una interacción física entre los objetos equivalente a una colisión real.



- ❑ La detección de colisiones es un problema complejo que requiere de diferentes algoritmos de técnicas para su correcta implementación.
- ❑ El caso más simple es el de una partícula así que vamos a empezar por ahí.
- ❑ Luego veremos algoritmos y métodos para sólido rígido.





Colisiones: Partículas





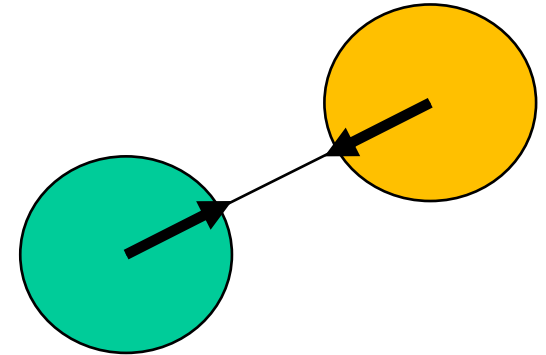
- ☐ Para saber como detectar colisiones de forma realista, primero tenemos que saber como se debería gestionar una colisión.
- ☐ Vamos a empezar por entender como se **gestionan las colisiones**.





- ❑ Para definir la colisión entre dos partículas hace falta definir sus **normales de contacto**.
- ❑ Cada partícula tiene una normal de contacto diferente y contraria a la de la otra.
- ❑ Se definen como vectores de módulo 1 cuya dirección es la dirección de colisión.
- ❑ Para la partícula b, la normal de colisión es:

$$\vec{n} = (\widehat{p_a - p_b})$$



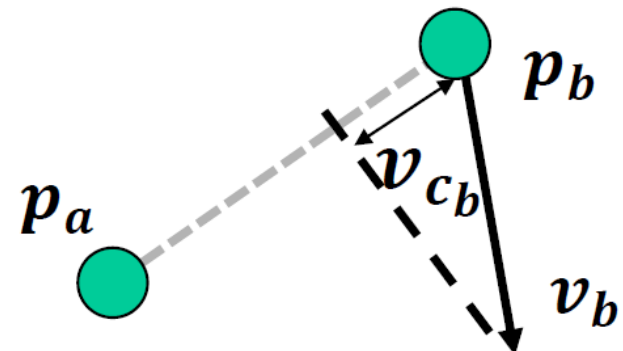
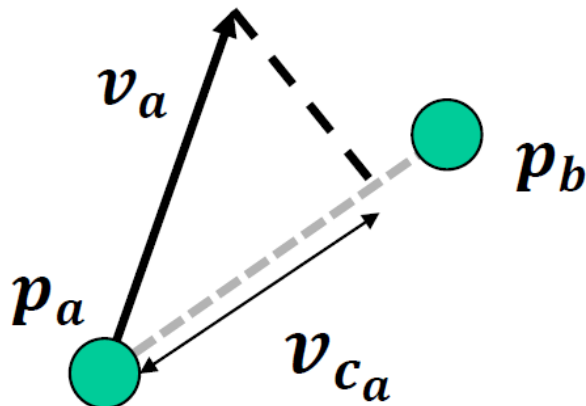
- ❑ Siendo p_a y p_b la posición de los centros.





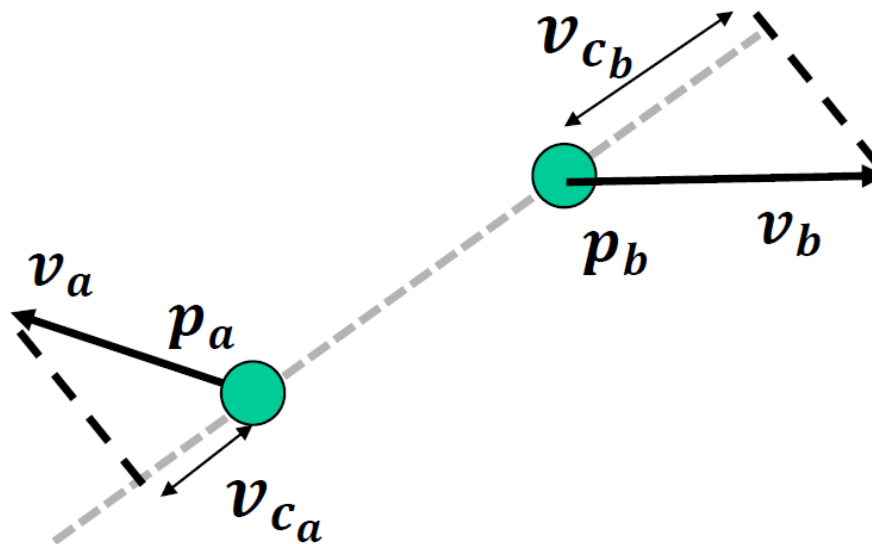
- ❑ **Closing Speed** (velocidad de acercamiento): Es la velocidad a la que dos objetos se acercan.
- ❑ Es un escalar: Producto escalar de las velocidades por la normal de contacto.
 - ❑ Si se están juntando será positiva
 - ❑ Si se están separando será negativa

$$v_c = v_a \cdot (\widehat{p_b - p_a}) + v_b \cdot (\widehat{p_a - p_b})$$





- ❑ **Separation Speed** (velocidad de acercamiento): Es la inversa de la velocidad de acercamiento.
- ❑ Si se están juntando, será negativo
- ❑ Si se están separando, será positivo





- ❑ La colisión pasa en un instante de tiempo.
 - ❑ Desde el punto de vista físico una interpenetración entre sólidos no deformables no debería de ocurrir.
- ❑ Manipularemos la velocidad y posición directamente mediante **impulsos** (variación instantánea de velocidad).
 - ❑ No lo haremos mediante fuerzas y aceleraciones para evitar interpenetraciones innecesarias.
 - ❑ Tenemos que seguir teniendo en cuenta la fuerza que lleva cada partícula para hacer colisiones realistas.





- ❑ Para resolver colisiones afectamos directamente a la velocidad
- ❑ No podemos usar fuerzas
 - ❑ Afectan a la aceleración
 - ❑ Necesitamos una respuesta instantánea
- ❑ El equivalente de la fuerza para velocidades se llama impulso

$$f = m \cdot a \quad \longleftrightarrow \quad g = m \cdot v$$

- ❑ g es la fuerza que lleva una partícula con masa m y velocidad v .
- ❑ También se conoce como **momento lineal** y con la letra p .





- ❑ Conceptos clave:
 - ❑ Velocidad de encuentro (closing velocity)
 - ❑ Velocidad de separación (separation velocity)
 - ❑ Normal de contacto
 - ❑ Momento líneal

- ❑ ¿Cómo se gestiona una colisión?
 - ❑ Conservación momentos: Para que una colisión sea realista los momentos totales de las partículas se deben de conservar.
 - ❑ Coeficiente de restitución





Conservación de momento

- En una colisión se conserva el momento total de los dos elementos
 - Mismo momento total en el instante 1 (pre-colisión) que en el instante 2 (post-colisión)

$$\underbrace{m_a v_{a_1} + m_b v_{b_1}}_{\text{pre-colisión}} = \underbrace{m_a v_{a_2} + m_b v_{b_2}}_{\text{post-colisión}}$$

- Con esta ecuación se modela el momento total
- No sabemos la velocidad resultante de cada objeto
 - Incógnitas: v_{a_2} y v_{b_2}





Conservación de momento

- Ambas partículas reciben el mismo impulso independientemente de la masa.

	Pre-colisión	Post-colisión	Impulso, g
Partícula A	$m_a \cdot v_{a1}$	$m_a \cdot v_{a2}$	$g_a = m_a \cdot (v_{a2} - v_{a1})$
Partícula B	$m_b \cdot v_{b1}$	$m_b \cdot v_{b2}$	$g_b = m_b \cdot (v_{b2} - v_{b1})$
total	$m_a \cdot v_{a1} + m_b \cdot v_{b1}$	$m_a \cdot v_{a2} + m_b \cdot v_{b2}$	$g_a + g_b = 0$

$$g_a = -g_b$$





- ❑ No siempre toda la energía se convierte en momento. Si tenemos perdidas el momento total disminuye.
- ❑ El **coeficiente de restitución (C)** nos define cuanto momento lineal (energía cinética) se mantiene.
 - ❑ Depende de los materiales de los objetos que colisionan
 - ❑ $C = 1.0 \rightarrow$ choque perfectamente elástico (la energía cinética se conserva totalmente) : Momento total antes y después de la colisión es el mismo.
 - ❑ $C = 0.0 \rightarrow$ choque perfectamente inelástico (la energía cinética deforma los objetos/aumenta temperatura...)





- Uniendo la ley de conservación del momento y la definición de coeficiente de restitución se puede llegar a una expresión para obtener el impulso sobre cada partícula:

$$g_a = -g_b = -\frac{(v_{a1} - v_{b1})(C + 1)}{\frac{1}{m_a} + \frac{1}{m_b}}$$





- ❑ Una fuerza calcula el valor total de la aceleración
 - ❑ Sin fuerzas no hay aceleración
- ❑ Un impulso calcula cómo se modifica la velocidad
 - ❑ Sin impulso la velocidad se mantiene constante
- ❑ No acumularemos impulsos sumándolos como hacemos con las fuerzas
- ❑ Los aplicaremos directamente cuando sucedan

$$v_2 = v_1 + \frac{1}{m} g$$





```
class ParticleContact
{
public:
    // Particles involved in the collision
    Particle* particle[2];

    // Restitution coefficient
    float restitution;

    // Contact normal,
    // from the first particle perspective
    Vector3 contactNormal;
};
```





```
class ParticleContact
```

```
{
```

```
    // Previous code
```

```
protected:
```

```
    // Resolves this contact for both velocity and interpenetration.
```

```
    void resolve(float t);
```

```
private:
```

```
    void resolveVelocity(float t);
```

```
};
```





```
void ParticleContact::resolve(float t)
```

```
{  
    resolveVelocity(t);  
    resolveInterpenetration(t);  
}
```

```
void ParticleContact::resolveVelocity(float t)
```

```
{
```

```
    // Compute impulse for each particle  $g_a = -g_b = -\frac{(v_{a1} - v_{b1})(C + 1)}{\frac{1}{m_a} + \frac{1}{m_b}}$   
    // Set new velocity for each particle
```

$$v_2 = v_1 + \frac{1}{m}g$$

```
}
```



Detección de colisiones



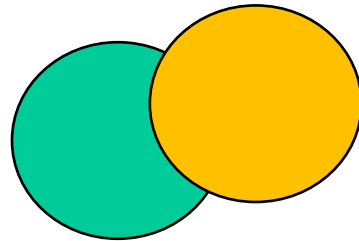


- ❑ Ahora que ya sabemos como gestionar colisiones, necesitamos crear un detector de colisiones.
- ❑ Será un módulo separado:
 - ❑ **Detección de colisión.**
 - ❑ Gestión de colisión.
- ❑ Detectar colisiones en una partícula es sencillo, sabiendo su geometría podemos saber fácilmente si dos partículas están colisionando.





- ❑ La **interpenetración** se produce cuando dos objetos están solapados.
- ❑ Hay una parte de un objeto que está dentro del otro objeto

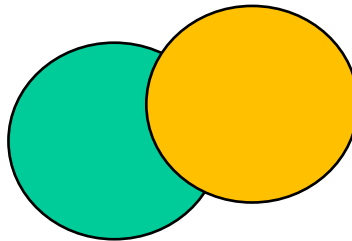


- ❑ No es suficiente con cambiar la velocidad
 - ❑ Podría ser pequeña y tardar en resolver la interpenetración.
- ❑ Hay que resolver la interpenetración como algo adicional y de manera “manual”.





- ❑ El sistema de colisiones debe detectar solapamientos
- ❑ Nos dará como resultado la intersección de los objetos
- ❑ La detección de colisiones nos devolverá el dato de nivel de interpenetración junto con el resto de la información de colisión





```
class ParticleContact  
{  
    // Previous code  
    float penetration;  
}
```





- ❑ Es una magnitud con signo: Es una distancia que se calcula con la dirección normal de colisión.
 - ❑ Si es menor que cero significa que los objetos están separados
 - ❑ Si es igual a cero significa que los objetos se están tocando
 - ❑ Si es mayor que cero significa que los objetos se solapan





Interpenetración (Un objeto)

- ❑ Para resolver el solapamiento debemos mover los objetos para que la distancia de interpenetración sea cero.
- ❑ Si solo tenemos un objeto es fácil
 - ❑ Una partícula colisionando con un muro estático
- ❑ Movemos la partícula en la dirección de la normal la distancia de interpenetración





Interpenetración (Dos objetos)

- ❑ Si tenemos dos objetos hay muchas opciones
- ❑ Podría mover cada partícula la mitad de la distancia
 - ❑ Conseguiría el objetivo → Resolver la interpenetración
 - ❑ Pero no es realista. Una partícula muy pesada solapando con otra poco pesada no debe moverse la misma distancia.
- ❑ ¿Cómo lo mejoramos?
- ❑ Aplicamos la distancia inversamente proporcional a la masa
 - ❑ El objeto de mayor masa se moverá menos
 - ❑ Interpolamos la distancia según su masa





Interpenetración (dos objetos)

- Podemos calcularlo mediante estas ecuaciones

$$\Delta \mathbf{p}_a = \frac{m_b}{m_a + m_b} d \vec{n} \quad \Delta \mathbf{p}_b = \frac{m_a}{m_a + m_b} d \vec{n}$$

- Es una simple interpolación lineal
- Donde, d es la distancia de interpenetración.





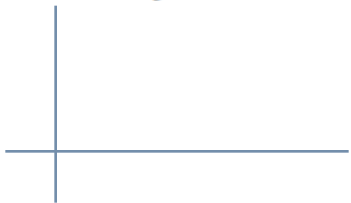
```
class ParticleContact
{
    // Previous code
private:
    void resolveInterpenetration(float t);
}

void ParticleContact::resolveInterpenetration(float t)
{
    // Check if there is interpenetration

    // Calculate factor of each particle depending on mass

    // Calculate the movement for each particle

    // Apply movement to the position of each particle
}
```



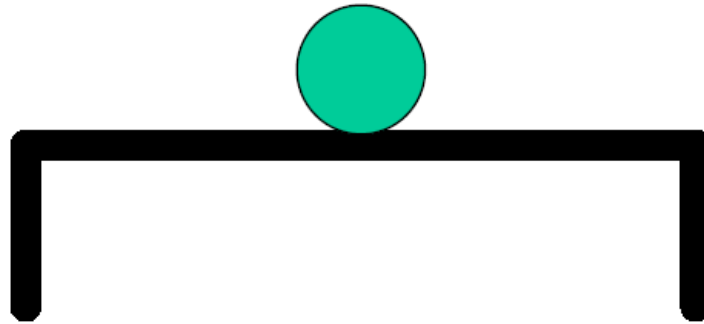


Colisiones estáticas

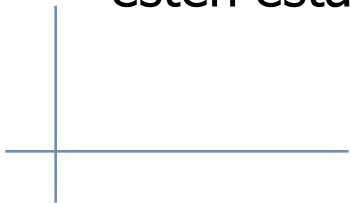




- Hay situaciones en donde tenemos colisiones, pero los objetos están en reposo uno con respecto del otro.

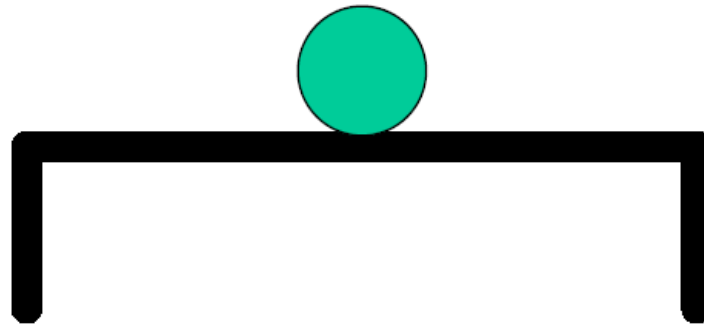


- Si utilizamos el algoritmo de colisión normal, se producirán vibraciones e interpenetraciones en bucle que evitan que los objetos estén estables.





- ❑ Necesitamos modificar nuestro algoritmo de colisiones para gestionar este tipo de situaciones.
- ❑ Lo que se suele hacer es poner límites de interpenetración, momentos lineales...
- ❑ Cuando dos partículas están estáticas y tienen valores de colisión pequeños se definen como partículas en **colisión estática** entre ellas y se gestionan de forma diferente.





Colisiones múltiples





- ☐ Si tenemos varias colisiones simultáneamente se pueden afectar entre ellas.
- ☐ Resolver una colisión puede modificar otra (cambiar interpenetración).
- ☐ La forma de proceder es resolver las colisiones de forma **iterativa**.
- ☐ Generación de lista de colisiones a procesar.





- ❑ Sistema de gestión de colisiones recibe una lista de colisiones.
- ❑ Las colisiones se resuelven individualmente y secuencialmente.
 - ❑ Resolver interpenetración
 - ❑ Calcular velocidad de colisión resultante (impulso)
- ❑ ¿Qué contactos resuelvo primero? ¿Da igual?



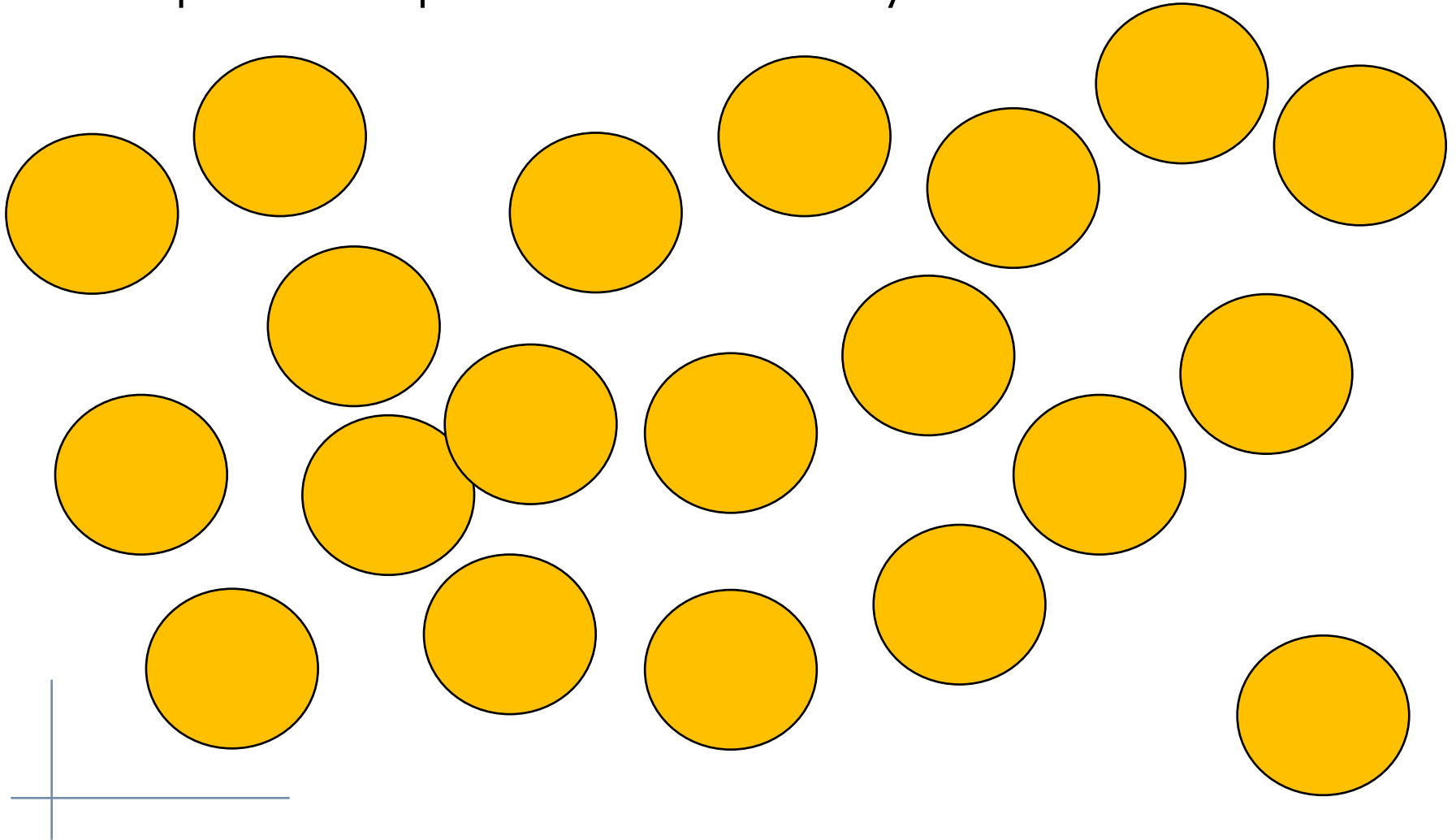


- ❑ En general lo mejor es resolver las colisiones importantes primero.
 - ❑ Colisiones con mucha interpenetración.
 - ❑ Colisiones con velocidades de acercamiento muy altas.
- ❑ Resolver las más importantes probablemente resuelva otras.
- ❑ Cada vez que resuelvo una colisión es necesario reevaluar el resto de colisiones.
- ❑ De esta forma, en general las colisiones se acaban resolviendo solas.
- ❑ Para evitar situaciones donde las colisiones entren en bucle, definimos un número máximo de iteraciones.



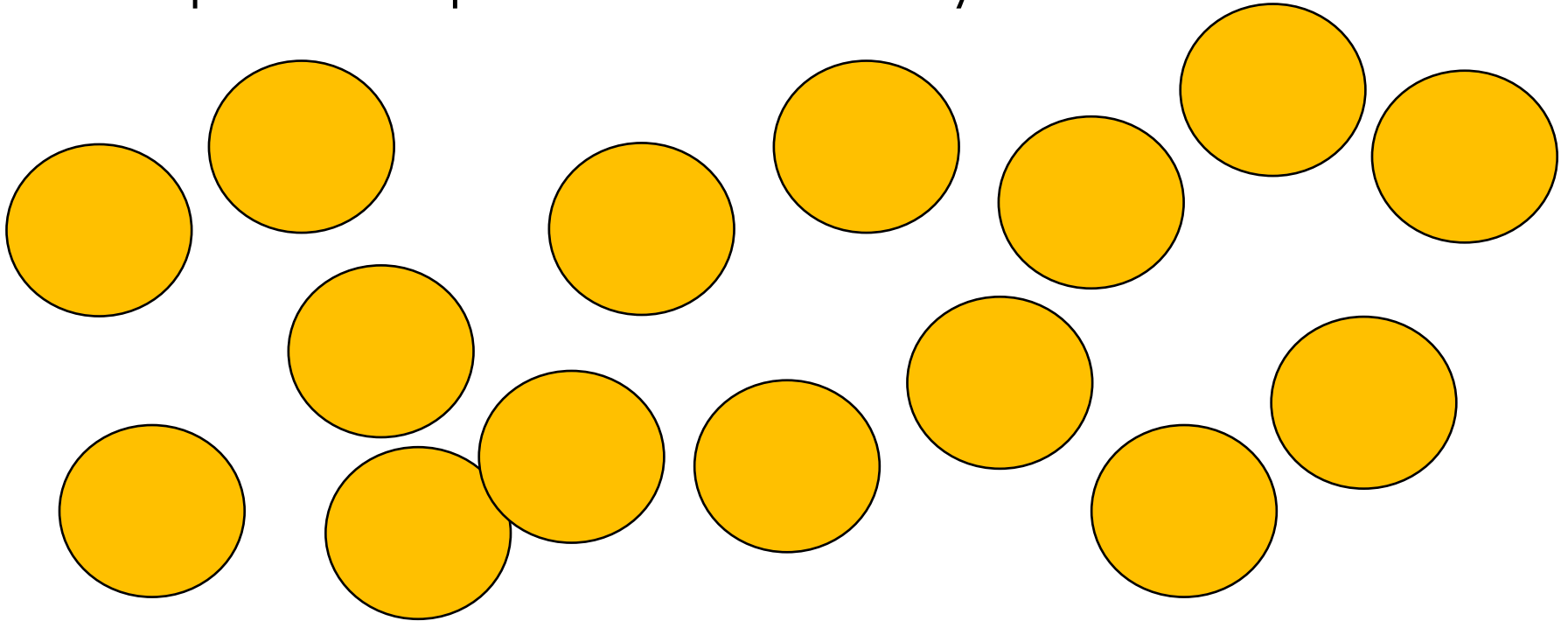


- ❑ ¿Qué pasa cuando tenemos muchas partículas? ¿Debemos de comprobar cada paso de simulación si hay colisiones entre ellas?





- ❑ ¿Qué pasa cuando tenemos muchas partículas? ¿Debemos de comprobar cada paso de simulación si hay colisiones entre ellas?



- ❑ Existen métodos para ahorrarnos comprobaciones innecesarias. Los veremos en el próximo tema.